

Unit – 1 : Session – 11 : SLO – 2 | Subroutine Overhead & Memory

Activity (Solutions)

- 1) Call-stack trace of recursion: `fact(3)` → pushes frame `fact(3)`; calls `fact(2)` → frame `fact(2)`; calls `fact(1)` → frame `fact(1)` returns 1; `fact(2)` returns $2*1=2$; `fact(3)` returns $3*2=6$ — each call pushes a new stack frame (parameters, locals, return address); returns pop them. Depth too large → `StackOverflowError`.
- 2) Track memory: `Runtime rt = Runtime.getRuntime(); long used = rt.totalMemory() - rt.freeMemory(); System.out.println("Used: " + used/1024 + " KB");` — measure before/after allocations.
- 3) Static vs dynamic creation: a static/class-level object is created once (loaded with the class) → small fixed footprint, shared by all instances; dynamic creation with `new` inside a loop allocates a fresh heap object per iteration → footprint grows with count until GC runs. E.g., `10000 new StringBuilder()` vs one reused instance — the static/reused one uses far less memory.